

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Аппаратно-программное обеспечение ЭВМ и сетей»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства
Intel – обработка символьных данных»

Выполнила:

Студентка 4 курса

Группы АС-66

Езепчук А.С.

Проверила:

Кулик А.Д.

Цель работы: изучение программной модели микропроцессора, практическая работа с подсистемой регистровой и оперативной памяти.

Вариант 6

Постановка задачи. Дан текст – непустая последовательность не длиннее ста символов. Признаком конца ввода является точка, в сам текст точка не входит.

Проверить, удовлетворяет ли текст заданному условию. Если условие выполнено, преобразовать текст по указанному в варианте правилу; в противном случае выдать соответствующее сообщение. Преобразованный текст напечатать.

Проверяемое условие и правила обработки текста определяются конкретным вариантом задания.

Если введенная последовательность символов не является текстом, преобразовывать ее не нужно, а следует напечатать соответствующее сообщение.

Ввод текста, проверка условия, обработка текста и печать результата должны выполняться последовательно, отдельными частями программы.

Используя знания и навыки, полученные в результате выполнения лабораторной работы №1, а также изучив дополнительные теоретические материалы, приведенные в методическом указании, написать программу на ассемблере, реализующую обработку текста согласно варианту, выданному преподавателем.

Проверяемое условие: Текст содержит равное количество прописных латинских и прописных русских букв.

Правило преобразования: Заменить каждую строчную латинскую букву соответствующей прописной буквой, а прописную – строчной.

Требования к программе: нельзя использовать дополнительную память, размер которой зависит от длины текста (в частности, стек) во всех вариантах, кроме тех, где явно не указано обратное.

При реализации вариантов, помеченных "с.к.", требуется использовать строковые команды с префиксами повторения.

Проверку условия и выполнение преобразований текста следует реализовать в виде процедур. Глобальные переменные не использовать.

Интерфейс программы должен быть достаточно удобным: надо печатать приглашение для ввода, пояснять, какое условие проверяется, истинно ли оно и как текст преобразуется.

Рекомендации: в некоторых вариантах удобно реализовать в качестве процедур или макросов проверку, является ли символ буквой, цифрой и т.п.

В процедуре преобразования текста может быть полезным иметь два указателя на позиции в тексте: откуда происходит чтение и куда идёт запись.

Максимально возможную длину текста описать в виде константы.

Код программы

```
.386
.model flat,stdcall
option casemap:none

includelib kernel32.lib

ExitProcess PROTO :DWORD
GetStdHandle PROTO :DWORD
ReadConsoleA PROTO :DWORD,:DWORD,:DWORD,:DWORD,:DWORD
WriteConsoleA PROTO :DWORD,:DWORD,:DWORD,:DWORD,:DWORD
SetConsoleCP PROTO :DWORD
SetConsoleOutputCP PROTO :DWORD

STD_INPUT_HANDLE EQU -10
STD_OUTPUT_HANDLE EQU -11
MAX_LEN EQU 100

.const

prompt db "Vvedite tekst (do 100 simvolov, konec - '.'): ",0
nottext db 13,10,"Vvedennaya posledovatelnost ne yavlyaetsya tekstom.",13,10,0
condition db 13,10,"Proveryaem uslovie: chislo propisnyh latinskih bukv ravno chislu propisnyh russkih bukv.",13,10,0
true_msg db "Uslovie vpolneno.",13,10,0
false_msg db "Uslovie ne vpolneno.",13,10,0
original db "Ishodny tekst: ",0
result db 13,10,"Preobrazovannyi tekst: ",0
newline db 13,10,0

.code

PrintString PROC
    push eax
    push ecx
    push edx
    push esi
    push edi

    mov esi,edx
    xor ecx,ecx

PRINT_LOOP:
    cmp byte ptr [esi],0
    je PRINT_DONE
    inc esi
    inc ecx
    jmp PRINT_LOOP

PRINT_DONE:
    push STD_OUTPUT_HANDLE
    call GetStdHandle
    mov edi,eax

    sub esp,4
    mov eax,esp

    push 0
    push eax
    push ecx
    push edx
    push edi
    call WriteConsoleA

    add esp,4

    pop edi
    pop esi
    pop edx
    pop ecx
    pop eax
    ret
```

PrintString ENDP

ReadText PROC

push ebp
mov ebp,esp
sub esp,16

push ebx
push esi
push edi

mov esi,edx
xor ebx,ebx
mov dword ptr [ebp-16],0

push STD_INPUT_HANDLE
call GetStdHandle
mov edi,eax

READ_LOOP:

lea edx,[ebp-4]

push 0
lea eax,[ebp-8]
push eax
push 1
push edx
push edi
call ReadConsoleA

test eax,eax
jz READ_ERROR

movzx eax,byte ptr [ebp-4]

cmp eax,'.'
je READ_END

cmp eax,13
je READ_ERROR

cmp eax,10
je READ_ERROR

cmp ebx,MAX_LEN
jae READ_OVERFLOW

mov byte ptr [esi],al
inc esi
inc ebx

jmp READ_LOOP

READ_OVERFLOW:

mov dword ptr [ebp-16],1
jmp READ_LOOP

READ_END:

cmp dword ptr [ebp-16],1
je READ_ERROR

mov byte ptr [esi],0
mov eax,ebx
jmp READ_EXIT

READ_ERROR:

mov eax,-1

READ_EXIT:

pop edi
pop esi
pop ebx

mov esp,ebp
pop ebp

```

    ret
ReadText ENDP

IsText PROC
    push ecx
    push edx
    push esi

    cmp eax,0
    je TEXT_ERROR

    mov ecx,eax
    mov esi,edx

TEXT_LOOP:
    movzx edx,byte ptr [esi]

    cmp edx,20h
    jb TEXT_ERROR

    cmp edx,7Fh
    je TEXT_ERROR

    inc esi
    dec ecx
    jnz TEXT_LOOP

    mov eax,1
    jmp TEXT_EXIT

TEXT_ERROR:
    xor eax,eax

TEXT_EXIT:
    pop esi
    pop edx
    pop ecx
    ret
IsText ENDP

CheckCondition PROC
    push ebx
    push ecx
    push esi
    push edi

    xor ebx,ebx
    xor edi,edi

    mov ecx,eax
    mov esi,edx

CONDITION_LOOP:
    movzx eax,byte ptr [esi]

    cmp eax,'A'
    jb CHECK_RUSSIAN
    cmp eax,'Z'
    ja CHECK_RUSSIAN

    inc ebx
    jmp CONDITION_NEXT

CHECK_RUSSIAN:
    cmp eax,0A8h
    je RUSSIAN_FOUND

    cmp eax,0C0h
    jb CONDITION_NEXT

    cmp eax,0DFh
    ja CONDITION_NEXT

RUSSIAN_FOUND:
    inc edi

```

```

CONDITION_NEXT:
    inc esi
    dec ecx
    jnz CONDITION_LOOP

    mov eax,ebx
    mov edx,edi

    pop edi
    pop esi
    pop ecx
    pop ebx
    ret
CheckCondition ENDP

TransformText PROC
    push ecx
    push esi

    mov ecx,eax
    mov esi,edx

TRANSFORM_LOOP:
    mov al,[esi]

    cmp al,'a'
    jb CHECK_UPPER
    cmp al,'z'
    ja CHECK_UPPER

    sub al,20h
    mov [esi],al
    jmp TRANSFORM_NEXT

CHECK_UPPER:
    cmp al,'A'
    jb TRANSFORM_NEXT
    cmp al,'Z'
    ja TRANSFORM_NEXT

    add al,20h
    mov [esi],al

TRANSFORM_NEXT:
    inc esi
    dec ecx
    jnz TRANSFORM_LOOP

    pop esi
    pop ecx
    ret
TransformText ENDP

start PROC
    push ebp
    mov ebp,esp
    sub esp,116

    push 1251
    call SetConsoleCP

    push 1251
    call SetConsoleOutputCP

    mov edx,OFFSET prompt
    call PrintString

    lea edx,[ebp-104]
    call ReadText

    cmp eax,-1
    je ERROR_TEXT

    mov [ebp-108],eax

```

```

    lea edx,[ebp-104]
    mov eax,[ebp-108]
    call IsText

    cmp eax,1
    jne ERROR_TEXT

    mov edx,OFFSET condition
    call PrintString

    lea edx,[ebp-104]
    mov eax,[ebp-108]
    call CheckCondition

    mov [ebp-112],eax
    mov [ebp-116],edx

    cmp eax,edx
    jne CONDITION_FALSE

    mov edx,OFFSET true_msg
    call PrintString

    mov edx,OFFSET original
    call PrintString

    lea edx,[ebp-104]
    call PrintString

    lea edx,[ebp-104]
    mov eax,[ebp-108]
    call TransformText

    mov edx,OFFSET result
    call PrintString

    lea edx,[ebp-104]
    call PrintString

    mov edx,OFFSET newline
    call PrintString

    push 0
    call ExitProcess

CONDITION_FALSE:
    mov edx,OFFSET false_msg
    call PrintString

    push 0
    call ExitProcess

ERROR_TEXT:
    mov edx,OFFSET nottext
    call PrintString

    push 0
    call ExitProcess

start ENDP

END start

```

Результат работы программы:

Vvedite tekst (do 100 simvolov, konec - '.'): 3aXoDit Ulitka v bar.

Proveryaem uslovie: chislo propisnyh latinskih bukv ravno chislu propisnyh russkih bukv.
Uslovie vpolneno.

Ishodny tekst: 3aXoDit Ulitka v bar

Preobrazovannyi tekst: 3aXodIT uLITKA V BAR

```
Vvedite tekst (do 100 simvolov, konec - '.'): Я aboobaboba.
```

```
Proveryaem uslovie: chislo propisnyh latinskih bukv ravno chislu propisnyh russkih bukv.  
Uslovie ne vpolneno.
```

```
Vvedite tekst (do 100 simvolov, konec - '.'): asdas
```

```
Vvedennaya posledovatelnost ne yavlyaetsya tekstom.
```

Вывод: в ходе лабораторной работы была изучена программная модель микропроцессора и получены практические навыки работы с регистрами процессора и оперативной памятью при разработке программы на языке ассемблера. В соответствии с заданным вариантом была разработана программа, выполняющая последовательный ввод текста, проверку корректности введенной последовательности, проверку условия равенства количества прописных латинских и прописных русских букв, а также преобразование текста при выполнении условия.